

Java에서의 Functional 프로그래밍

정진우



Functional 프로그래밍이란?

- 필수: Function이 first-class로 취급됨.
- Function이 value로 취급되며 argument 혹은 output으로 될 수 있음.
- Immutable data → Function이 pure하기 위한 조건. Referential transparency에 필요.
- Pure function → output 외 다른 effect가 없음. Side-effect에서 자유로움 (free)
- Referential transparency → Value가 실제 값으로 언제든지 변환이 가능

ref: https://en.wikipedia.org/wiki/Functional_programming



Functional 프로그래밍의 장점

- Immutable한 데이터 → Concurrent한 환경에서 data race 해결.
Thread-safety가 보장됨!
- Side-effect에서 자유로운 Pure function으로 수학 및 논리의 확인이 쉬움 → Haskell 등의 언어에서는 컴파일러에서 강제, Java 등에서는 확인이 어려움
- Haskell → 예쁨. 자연스러운 수학 및 함수의 표현이 가능.
NullPointerException 에러도 없음(Maybe 타입).

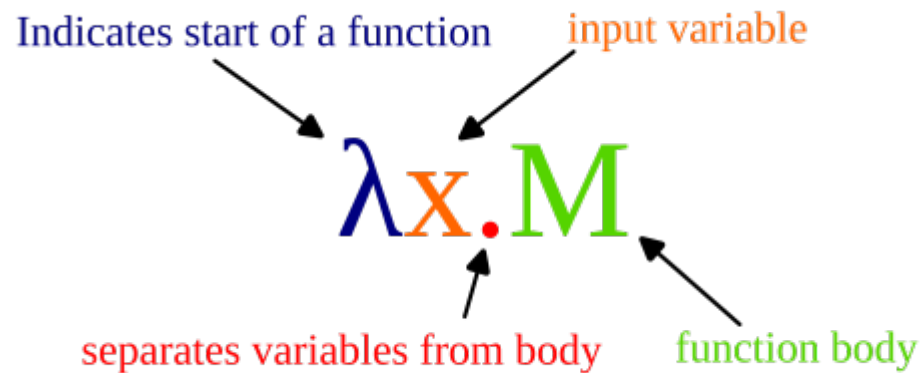


Lambda Calculus, Functional 프로그래밍의 기반

- 프로그래밍을 배웠으면 다들 개념적으로 아는 내용.
- 함수를 이용해 컴퓨터의 computation을 표현한 것. Untyped lambda calculus의 경우 computationally universal(i.e. Turing-complete, 모든 모델의 computation을 표현 가능)함.
- FP뿐만이 아니라 타입, 컴파일러 등 Programming Language Theory(PLT)의 기반



기초가 되는 Lambda term



이미지 출처:

https://en.wikipedia.org/wiki/Lambda_calculus#/media/File:LambdaAbstraction.svg

CC BY-SA 4.0

- $\lambda x.M \rightarrow$ Lambda Abstraction
 - λ : function의 시작,
 - x : variable $\rightarrow$ bound됨
 - M : function의 내용 (e.g, x , $x+1$)

Free variable, Substitution과 α -equivalence

Variable

- Free variable: bound 되지 않은 variable
- Bound variable: λ 로 bound 된 variable. e.g, $\lambda x.M$ 에서 M 의 x .
- Capture-avoiding substitution: $\lambda y.M [x : N]$ 의 형태로 표현. M 의 x 를 N 으로 변경. 이 경우 y 가 N 의 free variable set(i.e. $FV(N)$)에 속할 경우, alpha-renaming 진행
- α -equivalence는 bound variable을 다른 variable로 바꿔도 congruent(i.e. 같은 표현)이라는 것. α -conversion은 이러한 변환.

β -Reduction과 β -redex

$(\lambda f. \lambda g. \lambda x. f (g x)) (+1) (*2) 5$
 $\rightarrow_{\beta} (\lambda g. \lambda x. (+1) (g x)) (*2) 5$ $[f := (+1)]$
 $\rightarrow_{\beta} (\lambda x. (+1) ((*2) x)) 5$ $[g := (*2)]$
 $\rightarrow_{\beta} (+1) ((*2) 5)$ $[x := 5]$

이후 delta reduction을 통해 11이 됨



Haskell에 대해

- Functional Programming 언어. **Function이 first-class로 취급.**
- **Haskell의 function은 pure** 함(i.e. side-effect free) vs OCaml은 function이 side-effect가 가능(i.e. impurity 존재)
 - Referential Transparency가 가능
- **모든 것은 immutable** 함. Haskell의 List는 add등의 작업이 없음.
- Lazy evaluation → 꼭 필요할 때만 연산을 함 (무한한 sequence 역시 표현 가능 e.g, [1..]).
- Side-effect가 발생하는 대표인 I/O 역시 Monad로 value로 표현 됨.
- Type → class로 typeclass의 표현. Higher-kinded type(i.e. 다른 type에 대해 abstract하는 다른 type을 abstract할 수 있는 타입)의 표현이 가능.
- 강력한 type inference로 명시적으로 타입을 쓸 필요가 없음.
- 강한 type 시스템 + pure function + 엄격할 컴파일러 → 컴파일이 된다면 아마 잘 돌아감.



Lambda Expression의 언어별 표현

$(\lambda f. \lambda g. \lambda x. f (g x)) (+1) (*2) 5$

Java

```
import java.util.function.Function;
public class LambdaBetaReduction {
    static Function<Function<Integer,Integer>,
        Function<Function<Integer,Integer>,
            Function<Integer,Integer>>> compose =
        f -> g -> x -> f.apply(g.apply(x));

    public static void main(String[] args) {
        int result = compose.apply(n -> n + 1)
            .apply(n -> n * 2)
            .apply(5);
    }
}
```

Haskell

$(\lambda f \rightarrow \lambda g \rightarrow \lambda x \rightarrow f (g x)) (+1) (*2) 5$



Immutable한 데이터 및 pure function

Java

- record는 기본적으로 class만 immutable하며 내부 object는 재 reference만 불가하며 reference된 데이터는 변경 가능.
- List.copyOf 등으로 record 내부 데이터 역시 immutable하게 하기

Haskell

- Haskell의 경우 기본적으로 모든 데이터가 Immutable함. e.g, List의 경우 add, remove 등이 없음.



Java의 Optional과 Haskell의 Maybe

NullPointerException 에러 대신 없는 상태를 직접 핸들링

Java
Optional<A>

Haskell
data Maybe a = Nothing | Just a

Optional<Integer> → Optional.of(5) Maybe Int 필요한 경우 → Just 5

그런데 없는 경우 → Nothing

Optional<Integer> →
Optional.empty()

* FP가 아닌 Haskell이 처리하는 방식. Object의 reference가 null이 가능한 Java와 달리 Haskell에는 null이 없음.

Side-effect에 대해, I/O를 감싸기

Java

```
import java.util.Scanner;
import java.util.function.Function;
import java.util.function.Supplier;

public class PureIo {
    static Supplier<String> getLine = () -> new Scanner(System.in).nextLine();
    static Function<String, Supplier<Void>> putStrLn = s -> () ->
    { System.out.println(s); return null; };

    public static void main(String[] args) {
        putStrLn.apply("이름은??").get();
        String name = getLine.get();
        putStrLn.apply("내 이름은: " + name).get();
    }
}
```

Haskell

```
getLine :: IO String
putStrLn :: String -> IO ()

greet :: IO ()
greet = do
    putStrLn "이름은?"
    name <- getLine
    putStrLn ("내 이름은: " ++ name)
```



결론

- Functional 프로그래밍 구현
 - Java에서도 functional 프로그래밍이 가능함. 대신 boilerplate이 너무 많고 지저분함
 - Pure function → 비슷하게 흉내 가능. 다만 compiler에서 보장 x. 실제로 FP 언어들의 pure function과 비슷하게 가기는 쉽지 않음.
 - Maybe → Optional로 비슷하게 갈 수 있음. 다만 역시 compiler에서 강제 되는게 아님.
- 기타 Haskell의 feature 들과의 비교
 - Lazy evaluation → Stream으로 비슷하게 가능. 필요할 때 value 찾는건 비슷.
- 결론의 결론
 - Java는 Object-oriented 언어임.
 - FP와 Haskell의 기능을 모두 비교하기에는 양이 너무 많음
 - 구현이 됨을 보이는 건 단순하지만 어렵다는걸 주장하기에는 추가 조사가 많이 필요함



사용된 툴

- Java
 - 컴파일러: javac 25.0.3
 - Java 버전: openjdk 25.0.3
2026-04-21
- Haskell
 - 컴파일러: The Glorious Glasgow Haskell Compilation System, version 9.10.3



후속 연구 주제

- Java에서 부재한 Monad, typeclass, higher-kinded types에 대한 해결법
 - 외부 라이브러리인 Vavr을 이용한 Object-Functional 프로그래밍 분석
- Haskell에서의 다양한 syntax를 typed lambda calculus로 mapping 해 보기
- Java 컴파일러가 잡아주지 못 하는 side-effect에 대해, 기타 해결 방법 찾아보기
- OCaml과 Haskell의 비교

